Unit 4

Database Design

Informal Design Guidelines for Relational Databases

- What is relational database design?

  The grouping of attributes to form "good" relation schemas
- What are the criteria for "good" base relations?
- Informal guidelines that may be used as measures to determine the quality of relation schema design:

  * Making sure that the semantics of the attributes is clear in the schema
  * Reducing the redundant information in tuples
  * Reducing the NULL values in tuples
  * Disallowing the possibility of generating spurious tuples

Informal Design Guidelines for Relational Databases

1. Imparting Clear Semantics to Attributes in Relations

- Whenever we group attributes to form a relation schema, we assume that attributes belonging to one relation have certain real-world meaning and a proper interpretation associated with them.

For Example:

Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation.

Only foreign keys should be used to refer to other entities.

Entity and relationship attributes should be kept apart as much as possible.

Informal Design Guidelines for Relational Databases

1. Imparting Clear Semantics to Attributes in Relations

Guideline 1

“ *Design a relation schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types into a single relation.*

Intuitively, if a relation schema corresponds to one entity type or one relationship type, it is straightforward to interpret and to explain its meaning. Otherwise, if the relation corresponds to a mixture of multiple entities and relationships, semantic ambiguities will result and the relation cannot be easily explained. “

Informal Design Guidelines for Relational Databases

1. Imparting Clear Semantics to Attributes in Relations

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn

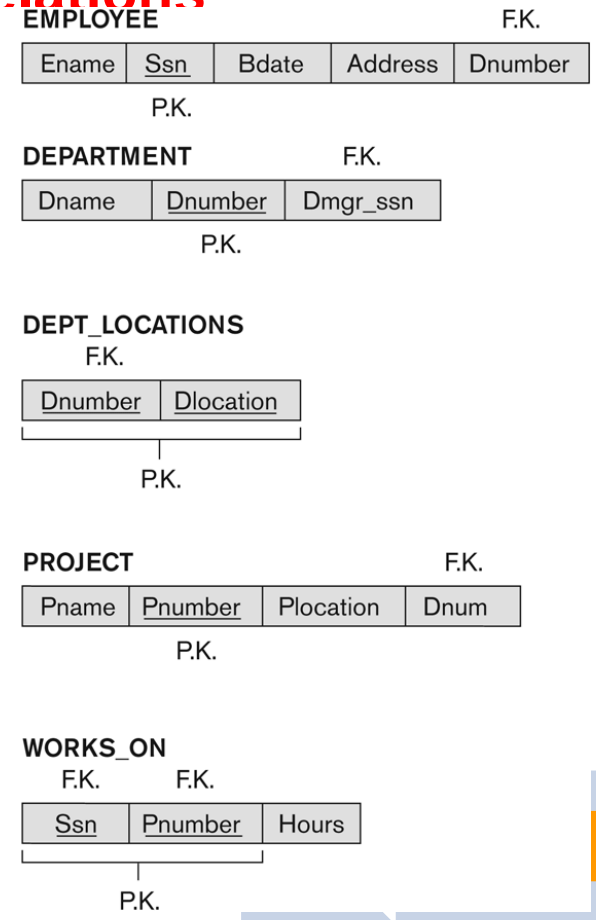


Figure 10.1
A simplified COMPANY relational database schema.

Informal Design Guidelines for Relational Databases

2. Redundant Information in Tuples and Update Anomalies

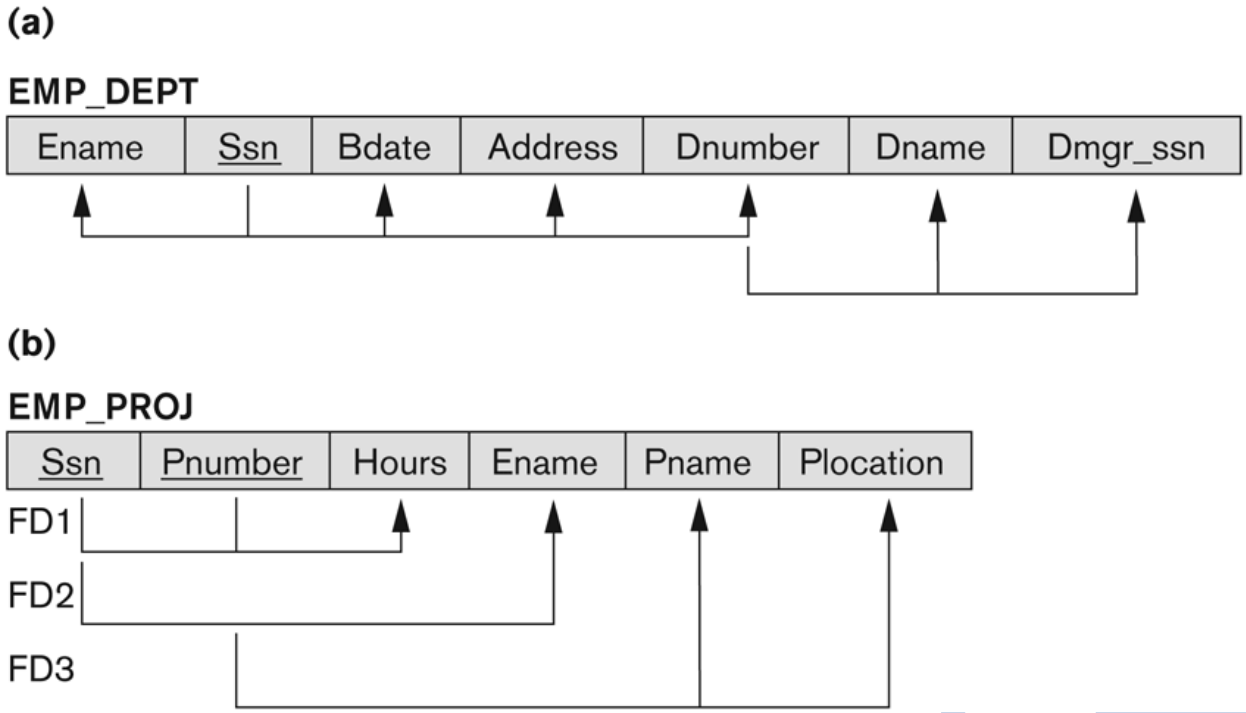
- One goal of schema design is to minimize the storage space used by the base relations.
- Storing natural joins of base relations leads to an additional problem referred to as update anomalies. These can be classified into

  * Insertion anomalies
  * Deletion anomalies
  * Modification anomalies

Informal Design Guidelines for Relational Databases

2. Redundant Information in Tuples and Update Anomalies

Figure 10.3
Two relation schemas suffering from update anomalies.
(a) EMP_DEPT and
(b) EMP_PROJ.



Informal Design Guidelines for Relational Databases

Figure 10.4
Example states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 10.2. These may be stored as base relations for performance reasons.

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Informal Design Guidelines for Relational Databases

2. Redundant Information in Tuples and Update Anomalies

Consider the relation: EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

Insert Anomaly:

- Cannot insert a project unless an employee is assigned to it. Conversely Cannot insert an employee unless an he/she is assigned to a project.

Delete Anomaly:

- When a project is deleted, it will result in deleting all the employees who work on that project. Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Informal Design Guidelines for Relational Databases

2. Redundant Information in Tuples and Update Anomalies

Consider the relation: EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)

Update Anomaly:

Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all employees working on project P1.

Informal Design Guidelines for Relational Databases

2. Redundant Information in Tuples and Update Anomalies

Guideline 2

“ *Design the base relation schemas so that no insertion, deletion, or modification anomalies are present in the relations. If any anomalies are present, note them clearly and make sure that the programs that update the database will operate correctly* “.

Informal Design Guidelines for Relational Databases

3. Null Values in Tuples

Problems with NULLs:

- Wastage of space
- Problems with understanding the meaning of the attributes
- Problems with specifying JOIN operations
- Applying aggregate operations

Reasons for Nulls:

- Attribute not applicable or invalid
- Attribute value unknown (may exist)
- Value known to exist, but unavailable

Informal Design Guidelines for Relational Databases

3. Null Values in Tuples

Guideline 3

- “ *As far as possible, avoid placing attributes in a base relation whose values may frequently be NULL . If NULL s are unavoidable, make sure that they apply in exceptional cases only and do not apply to a majority of tuples in the relation.* “

Informal Design Guidelines for Relational Databases

- **4. Generation of Spurious Tuples**

Bad designs for a relational database may result in erroneous results for certain JOIN operations

Guideline 4

“ Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (primary key, foreign key) pairs in a way that guarantees that no spurious tuples are generated. Avoid relations that contain matching attributes that are not (foreign key, primary key) combinations because joining on such attributes may produce spurious tuples. “

Functional Dependency

- Functional Dependency (FD) is a constraint that determines the relation of one attribute to another attribute in a Database Management System (DBMS).
- Functional Dependency helps to maintain the quality of data in the database.
- It plays a vital role to find the difference between good and bad database design.
- The functional dependency is a relationship that exists between two attributes.

Functional Dependency

- It typically exists between the primary key and non-key attribute within a table.
- A functional dependency is denoted by an arrow "→".
- The functional dependency of X on Y is represented by X → Y.
- The left side of FD is known as a determinant, the right side of the production is known as a dependent.

Functional Dependency

Example 1: Assume we have an employee table with attributes: Emp_Id, Emp_Name, Emp_Address.

- Here Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it.
- Functional dependency can be written as: Emp_Id → Emp_Name

Example 2: Employee (Enumber, Ename, Salary, City)

- In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc. By this, we can say that the city, Employee Name, and salary are functionally dependent on Employee number.

Functional Dependency

Example 3:

From the table we can conclude some valid functional dependencies:

roll_no	name	dept_name	dept_building
42	abc	CO	A4
43	pqr	IT	A3
44	xyz	CO	A4
45	xyz	IT	A3
46	mno	EC	B2
47	jkl	ME	B2

Functional Dependency

Example 3: Valid Functional Dependencies

- roll_no → { name, dept_name, dept_building }, Here, roll_no can determine values of fields name, dept_name and dept_building, hence a valid Functional dependency.
- roll_no → dept_name , Since, roll_no can determine whole set of {name, dept_name, dept_building}, it can determine its subset dept_name also.
- dept_name → dept_building , Dept_name can identify the dept_building accurately, since departments with different dept_name will also have a different dept_building.
- roll_no → name,

Functional Dependency

Example 3: Invalid Functional Dependencies

- name → dept_name, Students with the same name can have different dept_name, hence this is not a valid functional dependency.
- dept_building → dept_name, There can be multiple departments in the same building, For example, in the above table departments ME and EC are in the same building B2, hence dept_building → dept_name is an invalid functional dependency.
- name → roll_no,
- {name, dept_name} → roll_no,
- dept_building → roll_no, etc.

Advantages of Functional Dependency

- Functional Dependency avoids data redundancy. Therefore same data do not repeat at multiple locations in that database
- It helps you to maintain the quality of data in the database
- It helps you to defined meanings and constraints of databases
- It helps you to identify bad designs
- It helps you to find the facts regarding the database design

Types of Functional dependency

Functional Dependency has four forms:

* Trivial Functional Dependency
* Non-Trivial Functional Dependency
* Multivalued Functional Dependency
* Transitive Functional Dependency

Types of Functional dependency

1. Trivial Functional Dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant.
- i.e. If X → Y and Y is the subset of X, then it is called trivial functional dependency

  Example: Student(roll_no, name, age)
- Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name}
- Similarly, roll_no → roll_no is also an example of trivial functional dependency.

Types of Functional dependency

2. Non-Trivial Functional Dependency

- In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant.
- i.e. If X → Y and Y is not a subset of X, then it is called Non-trivial functional dependency.

  Example: Student(roll_no, name, age)
- Here, roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no.
- Similarly, {roll_no, name} → age is also a non-trivial functional dependency, since age is not a subset of {roll_no, name}.

Types of Functional dependency

3. Multivalued Functional Dependency

- In Multivalued functional dependency, entities of the dependent set are not dependent on each other.
- i.e. If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.

  Example: Student(roll_no, name, age)
- Here, roll_no → {name, age} is a multivalued functional dependency, since the dependents name & age are not dependent on each other(i.e. name → age or age → name doesn't exist !)

Types of Functional dependency

4 . Transitive Functional Dependency

- In transitive functional dependency, dependent is indirectly dependent on determinant.
- i.e. If a → b & b → c, then according to axiom of transitivity, a → c. This is a transitive functional dependency

  Example: Customer(enroll_no, name, dept, building_no)
- Here, enrol_no → dept and dept → building_no,
- Hence, according to the axiom of transitivity, enrol_no → building_no is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Inference Rule (IR)

- The inference rule is a type of assertion. It can apply to a set of FD(functional dependency) to derive other FD.
- Using the inference rule, we can derive additional functional dependency from the initial set.
- To determine a systematic way to infer dependencies, we must discover a set of inference rules that can be used to infer new dependencies from a given set of dependencies.
- The Armstrong's axioms are the basic inference rule.
- Armstrong's axioms are used to conclude functional dependencies on a relational database.

Inference Rule (IR)

- The term Armstrong axioms refer to the sound and complete set of inference rules or axioms, introduced by William W. Armstrong, that is used to test the logical implication of functional dependencies.
- If F is a set of functional dependencies then the closure of F, denoted as F^+, is the set of all functional dependencies logically implied by F.
- Armstrong's Axioms are a set of rules, that when applied repeatedly, generates a closure of functional dependencies.

Six Inference Rules

1. Reflexive Rule (IR1) (Axiom of reflexivity):

If Y is a subset of X, then X determines Y.

I.e If $X \supseteq Y$ then $X \rightarrow Y$

- Example:

  X = {a, b, c, d, e}

  Y = {a, b, c}

  Here Y is a subset of X, and therefore $X \rightarrow Y$

Six Inference Rules

2. Augmentation Rule (IR2) (Axiom of augmentation):

The augmentation is also called as a partial dependency. In augmentation, if X determines Y, then XZ determines YZ for any Z.

I.e If X → Y then XZ → YZ

- Example:

  For R(ABCD), if A → B then AC → BC.

Six Inference Rules

3. Transitive Rule (IR3) (Axiom of transitivity):

In the transitive rule, if X determines Y and Y determine Z, then X must also determine Z.

I.e If X → Y and Y → Z then X → Z

- Example:

  Customer(enroll_no, name, dept, building_no)

- Here, if enrol_no → dept and dept → building_no then enrol_no → building_no is a valid functional dependency

Six Inference Rules

Secondary Rules (These rules can be derived from the above axioms.)

4. Union Rule (IR4):

Union rule says, if X determines Y and X determines Z, then X must also determine Y and Z. I.e If X → Y and X → Z then X → YZ

Proof:

X → Y (given) …. (1)

X → Z (given) …. (2)

X → XY (using Rule 2 on (1) by augmentation with X) …. (3)

XY → YZ (using Rule 2 on (2) by augmentation with Y) …. (4)

X → YZ (using Rule 3 on (3) and (4))

Six Inference Rules

Secondary Rules (These rules can be derived from the above axioms.)

5. Decomposition Rule (IR5):

Decomposition rule is also known as project rule. It is the reverse of union rule.

- This Rule says, if X determines Y and Z, then X determines Y and X determines Z separately. I.e If X → YZ then X → Y and X → Z .

Proof:

X → YZ (given)…. (1)

YZ → Z and YZ → Y (using reflexive method) …. (2)

X → Y and X → Z (using transitive rule) …. (3)

Six Inference Rules

Secondary Rules (These rules can be derived from the above axioms.)

6. Pseudo transitive Rule (IR6):

In Pseudo transitive Rule, if X determines Y and YZ determines W, then XZ determines W. I.e If X → Y and YZ → W then XZ → W.

Proof:

1. X → Y (given)
2. YZ → W (given)
3. XZ → YZ (using IR2 on 1 by augmenting with Z)
4. XZ → W (using IR3 on 3 and 2)

Six Inference Rules

Example:

- 1. If X = Firstname, Lastname and Y= Lastname then X → Y.
- 2. If Regno → Firstname, Lastname then,

  Regno, address → Firstname, Lastname, address.
- 3. If Rollno → name and Rollno → address then, Rollno → name, address.
- 4. If Rollno → name and name, marks → percentage then,

  Rollno,marks → percentage.
- 5. If Rollno → address and address → Pincode then Rollno → Pincode.
- 6. If Rollno → Firstname, Lastname then

  Rollno → Firstname and Rollno → Lastname.

Closure of an Attribute Set

- The set of all those attributes which can be functionally determined from an attribute set is called as a closure of that attribute set or
- Closure of an attribute x is the set of all attributes that are functional dependencies on X with respect to F.
- Closure of attribute set {X} is denoted as $\{X\}^+$ which means what X can determine.
- Closure of a set F of FDs is the set F^+ of all FDs that can be inferred from F.

Algorithm of Determining X+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;

repeat

    $oldX^+ := X^+$;

    for each functional dependency $Y \rightarrow Z$ in F do

        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;

    until ($X^+ = oldX^+$);

Finding the Keys Using Closure

Super Key:

- If the closure result of an attribute set contains all the attributes of the relation, then that attribute set is called as a super key of that relation.
- "The closure of a super key is the entire relation schema."

Candidate Key:

- If there exists no subset of an attribute set whose closure contains all the attributes of the relation, then that attribute set is called as a candidate key of that relation.

"*Candidate key of R: X is a candidate keys of R if X->{R}*"

Determining X+, the Closure of X under F- Example

Consider a relation FD set for relation STUDENT

- F= {STUD_NO->STUD_NAME,
  STUD_NO->STUD_PHONE,
  STUD_NO->STUD_STATE,
  STUD_NO->STUD_COUNTRY,
  STUD_NO -> STUD_AGE,
  STUD_STATE->STUD_COUNTRY }
- Using FD set of STUDENT, attribute closure can be determined as:
- (STUD_NO)+ = {STUD_NO, STUD_NAME, STUD_PHONE,
  STUD_STATE, STUD_COUNTRY, STUD_AGE}
- (STUD_STATE)+ = {STUD_STATE, STUD_COUNTRY}.

Determining X+, the Closure of X under F- Example

(STUD_NO, STUD_NAME)+ = {STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_COUNTRY, STUD_AGE}

(STUD_NO)+= {STUD_NO, STUD_NAME, STUD_PHONE, STUD_STATE, STUD_COUNTRY, STUD_AGE}

(STUD_NO, STUD_NAME) will be super key but not candidate key because its subset (STUD_NO)+ is equal to all attributes of the relation. So, STUD_NO will be a candidate key.

Determining X+, the Closure of X under F- Example

Consider a relation R (A , B , C , D , E , F , G) with the functional dependencies- A → BC, BC → DE, D → F, CF → G

- To find attribute closure of an attribute set:

  * Add elements of attribute set to the result set.

  * Recursively add elements to the result set which can be functionally determined from the elements of the result set.

Determining X+, the Closure of X under F- Example

Closure of attribute A-

$A^+ = \{A\}$

$= \{A, B, C\}$ (Using $A \rightarrow BC$)

$= \{A, B, C, D, E\}$ (Using $BC \rightarrow DE$)

$= \{A, B, C, D, E, F\}$ (Using $D \rightarrow F$)

$= \{A, B, C, D, E, F, G\}$ (Using $CF \rightarrow G$)

Thus,

$A^+ = \{A, B, C, D, E, F, G\}$

Determining X+, the Closure of X under F- Example

Closure of attribute D-

$D^+ = \{ D \}$

$= \{ D , F \}$ (Using $D \rightarrow F$)

We can not determine any other attribute using attributes D and F contained in the result set.

Thus,

$$D^+ = \{ D , F \}$$

Determining X+, the Closure of X under F- Example

Closure of attribute set {B, C}-

$\{B, C\}^+ = \{B, C\}$

$= \{B, C, D, E\}$ (Using $BC \rightarrow DE$)

$= \{B, C, D, E, F\}$ (Using $D \rightarrow F$)

$= \{B, C, D, E, F, G\}$ (Using $CF \rightarrow G$)

Thus,

$\{B, C\}^+ = \{B, C, D, E, F, G\}$

$\{CF\}^+ = \{C, F, G\}$

Determining X+, the Closure of X under F- Example

Super key and Candidate Keys

- The closure of attribute A is the entire relation schema. Thus, attribute A is a super key for that relation.
- No subset of attribute A contains all the attributes of the relation. Thus, attribute A is also a candidate key for that relation.

Determining X+, the Closure of X under F- Example

Example-2 : Consider another relation R(A, B, C, D, E) having the Functional dependencies :

FD1 : A → BC

FD2 : C → B

FD3 : D → E

FD4 : E → D

Determining X+, the Closure of X under F- Example

$\{A\}^+ = \{A, B, C\}$

$\{B\}^+ = \{B\}$

$\{C\}^+ = \{C, B\}$

$\{D\}^+ = \{E, D\}$

$\{E\}^+ = \{E, D\}$

Determining X+, the Closure of X under F- Example

In this case, a single attribute is unable to determine all the attribute on its own. we need to combine two or more attributes to determine the candidate keys.

$$\{A, D\}^{+} = \{A, B, C, D, E\}$$

$$\{A, E\}^{+} = \{A, B, C, D, E\}$$

Determining X+, the Closure of X under F- Example

Consider a relation R(A,B,C,D,E,F)

F: E->A, E->D, A->C, A->D, AE->F, AG->K. Find the closure of E or E+

Solution

The closure of E or E+ is as follows −

E+ = E

=EA {for E->A add A}

=EAD {for E->D add D}

=EADC {for A->C add C}

=EADC {for A->D D already added}

=EADCF {for AE->F add F}

=EADCF {for AG->K don't add k AG $\not\subset$ D+)

Determining X+, the Closure of X under F- Example

R(A,B,C,D,E,F) WHERE

F:A->BC, B->D, C->DE, BC->F. Then, find the candidate keys of R.

Solution

A+= {A,B,C,D,E,F}={R}=>A is a candidate key

B+= {B,D} => B is not a candidate key

C+= {C,D,E} => C is not a candidate key

BC+= {B,C,D,E,F} => BC is not a candidate key

Normalization

A large database defined as a single relation may result in data duplication. This repetition of data may result in:

- Making relations very large.
- It isn't easy to maintain and update data as it would involve searching many records in relation.
- Wastage and poor utilization of disk space and resources.
- The likelihood of errors and inconsistencies increases.

So to handle these problems, we should analyze and decompose the relations with redundant data into smaller, simpler, and well-structured relations that are satisfy desirable properties. Normalization is a process of decomposing the relations into relations with fewer attributes.

What is Normalization?

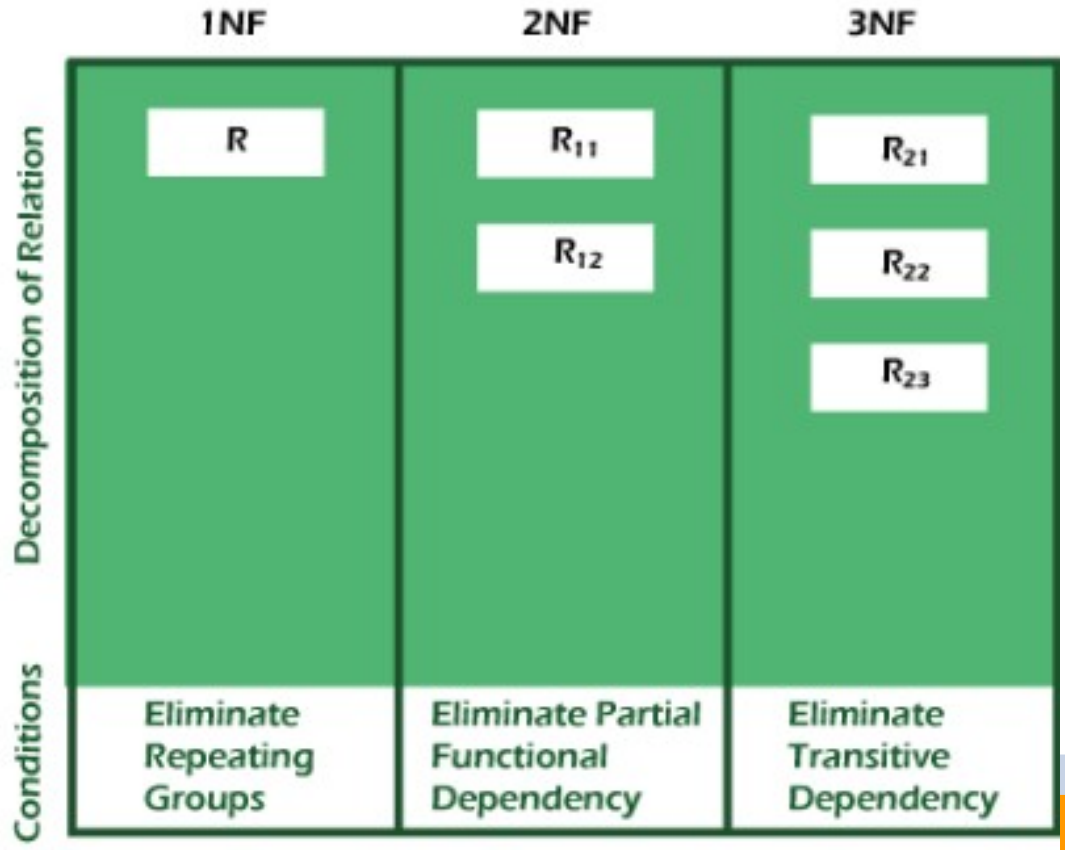
- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

“ Normalization is the process of minimizing redundancy from a relation or set of relations. Redundancy in relation may cause insertion, deletion, and update anomalies. “

Types of Normal Forms:

- Normalization works through a series of stages called Normal forms.
- The normal forms apply to individual relations.
- The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:



Types of Normal Forms:

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.

Types of Normal Forms:

First Normal Form (1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. i.e No multivalued columns are allowed.
- No repeating columns within a row.
- It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations.

Types of Normal Forms:

First Normal Form (1NF)

Example: Employee (unnormalized)

emp_no	name	dept_no	dept_name	skills
1	Kevin Jacobs	201	R&D	C, Perl, Java
2	Barbara Jones	224	IT	Linux, Mac
3	Jake Rivera	201	R&D	DB2, Oracle, Java

Types of Normal Forms:

First Normal Form (1NF)

Example: Employee (1NF)

emp_no	name	dept_no	dept_name	skills
1	Kevin Jacobs	201	R&D	C
1	Kevin Jacobs	201	R&D	Perl
1	Kevin Jacobs	201	R&D	Java
2	Barbara Jones	224	IT	Linux
2	Barbara Jones	224	IT	Mac
3	Jake Rivera	201	R&D	DB2
3	Jake Rivera	201	R&D	Oracle
3	Jake Rivera	201	R&D	Java

Types of Normal Forms:

Second Normal Form (2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on the primary key.
- Any non-dependent attributes are moved into a smaller (subset) table.
- 2NF improves data integrity.
- Prevents update, insert, and delete anomalies.

Types of Normal Forms:

Second Normal Form (2NF)

- Name, dept_no, and dept_name are functionally dependent on emp_no. i.e emp_no → name, dept_no, dept_name
- Skills is not functionally dependent on emp_no since it is not unique to each emp_no. Move this attribute to new relation.

Types of Normal Forms:

Second Normal Form (2NF)

Employee (2NF)

emp_no	name	dept_no	dept_name
1	Kevin Jacobs	201	R&D
2	Barbara Jones	224	IT
3	Jake Rivera	201	R&D

Skills

emp_no	skills
1	C
1	Perl
1	Java
2	Linux
2	Mac
3	DB2
3	Oracle
3	Java

Types of Normal Forms:

Third Normal Form (3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form.

    * Transitive dependence - two separate entities exist within one table.
    * Any transitive dependencies are moved into a smaller (subset) table.

Types of Normal Forms:

Third Normal Form (3NF)

Dept_no and dept_name are functionally dependent on emp_no however, department can be considered a separate entity. Move this to a seperate table.

Employee (3NF)

emp_no	name	dept_no
1	Kevin Jacobs	201
2	Barbara Jones	224
3	Jake Rivera	201

Dept (3NF)

dept_no	dept_name
201	R&D
224	IT

Lossless Decomposition

> Lossless join decomposition is a decomposition of a relation R into relations R1, R2 such that if we perform a natural join of relation R1 and R2, it will return the original relation R.

>This is effective in removing redundancy from databases while preserving the original data.

>In other words by lossless decomposition, it becomes feasible to reconstruct the relation R from decomposed tables R1 and R2 by using Joins.

>Only 1NF,2NF,3NF and BCNF are valid for lossless join decomposition.

Lossless Decomposition

> In Lossless Decomposition, we select the common attribute and the criteria for selecting a common attribute is that the common attribute must be a candidate key or super key in either relation R1, R2, or both.

>Decomposition of a relation R into R1 and R2 is a lossless-join decomposition if at least one of the following functional dependencies are in F+ (Closure of functional dependencies)

$R1 \cap R2 \rightarrow R1$

OR

$R1 \cap R2 \rightarrow R2$

Lossless Decomposition

> So in simple terms,

>Decomposition is said to be lossy, if it is impossible to reconstruct the original table from the decomposed tables without loss of information.

>Decomposition is said to be lossless, if it is possible to reconstruct the original table by using a natural join without any loss of information.

Lossless Decomposition

Let us see an example –

<EmpInfo>

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Decompose the above table into two tables:

Lossless Decomposition

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Emp_ID	Dept_Name
Dpt1	E001	Operations
Dpt2	E002	HR
Dpt3	E003	Finance

Lossless Decomposition

Now, Natural Join is applied on the above two tables –

The result will be –

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Therefore, the above relation had lossless decomposition i.e. no loss of information.

Lossless Decomposition

If we Decompose the above table into two tables −

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Dept_Name
Dpt1	Operations
Dpt2	HR
Dpt3	Finance

Now, you won't be able to join the above tables, since Emp_ID isn't part of the DeptDetails relation.

Therefore, the above relation has lossy decomposition.

Steps to check for Lossless Decomposition

- **Step 1** – Create a table with M rows and N columns
  - M= number of decomposed relations.
  - N= number of attributes of original relation.
- **Step 2** – If a decomposed relation Ri has attribute A then
- Insert a symbol (say 'a') at position (Ri,A)
- **Step 3** – Consider each FD X->Y
  - If column X has two or more symbols then

    Insert symbols in the same place (rows) of column Y.
- **Step 4** – If any row is completely filled with symbols then
  - Decomposition is lossless.
  - Else
    - Decomposition is lossy.

Algorithm 15.2 Testing for the lossless (nonadditive) join property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, ..., R_m\}$ of R, and a set F of functional dependencies.

1. Create an initial matrix S with one row i for each relation R_i in D, and one column j for each attribute A_j in R.
2. Set $S(i,j) := b_{ij}$ for all matrix entries.
   (* each b_{ij} is a distinct symbol associated with indices (i,j) *)
3. For each row i representing relation schema R_i
   {for each column j representing attribute A_j
   {if (relation R_i includes attribute A_j) then set $S(i,j) := a_j$;};};
   (* each a_j is a distinct symbol associated with index (j) *)
4. Repeat the following loop until a *complete loop execution* results in no changes to S
   {for each functional dependency $X \rightarrow Y$ in F
   {for all rows in S *which have the same symbols* in the columns corresponding to attributes in X
   {make the symbols in each column that correspond to an attribute in Y be the same in all these rows as follows: if any of the rows has an "a" symbol for the column, set the other rows to that *same* "a" symbol in the column. If no "a" symbol exists for the attribute in any of the rows, choose one of the "b" symbols that appear in one of the rows for the attribute and set the other rows to that same "b" symbol in the column ;};};};
5. If a row is made up entirely of "a" symbols, then the decomposition has the lossless join property; otherwise it does not.

Lossless Decomposition

Problem

Consider an example to check whether the given relation can be decomposed with lossy or lossless by applying the algorithm.

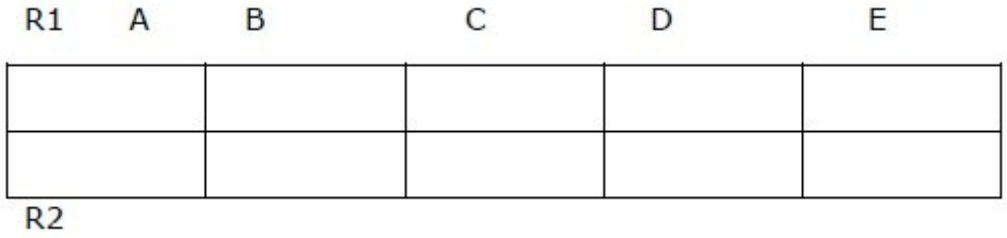
Consider R(A,B,C,D,E)

F:{A->B, BC->E, ED->A}

R is decomposed into R1(A,B) and R2(A,C,D,E). Check the decomposition is lossy or lossless.

Lossless Decomposition

Step 1



Step 2

R1	A	B	C	D	E
	a	a			
	a		a	a	a
R2					

Step 3 Now let us insert symbol ‘a’ for every FD, in second row

R1	A	B	C	D	E
	a	a			
	a	a	a	a	a
R2					

R2 is completely filled => decomposition is lossless.

Lossless Decomposition

Consider the following relations:

R={ssn, ename, pno, pname, ploc, hours}

R1={ssn, ename} R2={pno, pname, ploc} R3={ssn, pno, hours}

Check whether the decomposition D={ R1, R2, R3 } is lossless or not for the following functional dependencies: F = { ssn → ename, pno →{pname, ploc}, {ssn, pno} →hours} **4**

	Ssn	Ename	Pno	Pname	Ploc	Hours
R1	a1	a2				
R2			a3	a4	a5	
R3	a1	a2	a3	a4	a5	a6

Last row is entirely made up of “a” symbols. Therefore the decomposition is lossless.

Lossless Decomposition

1. Given a relational schema R = { SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS } and the decomposed table are

R1 = { SSN, ENAME }

R2 = { PNUMBER, PNAME, PLOCATION }

R3 = { SSN, PNUMBER, HOURS }

FD = { SSN → ENAME, PNUMBER → { PNAME, PLOCATION}, { SSN, PNUMBER } → HOURS }.

Identify whether the given decomposition of R, into R1, R2, and R3 is lossless or lossy decomposition?

Lossless Decomposition

2. Given a relational schema R = { SSN, ENAME, PNUMBER, PNAME, PLOCATION, HOURS } and the decomposed table

R1 = { ENAME,PLOCATION }

R2 = { SSN,PNUMBER, PNAME, PLOCATION,HOURS }

FD = { SSN → ENAME, PNUMBER → { PNAME, PLOCATION}, { SSN, PNUMBER } → HOURS }.

Identify whether the given decomposition of R, into R1, R2, and R3 is lossless or lossy decomposition?

3. Given a relational schema R ={A1,A2,A3,A4,A5}

R1={A1,A2,A3,A5}, R2={A1,A3,A4} and R3={A4,A5}

FD= { A1→{A3,A5}, A5 →{A1,A4}, {A3,A4} →A2

Identify whether the given decomposition of R, into R1, R2, and R3 is lossless or lossy decomposition?

Unit 4

Transaction Processing

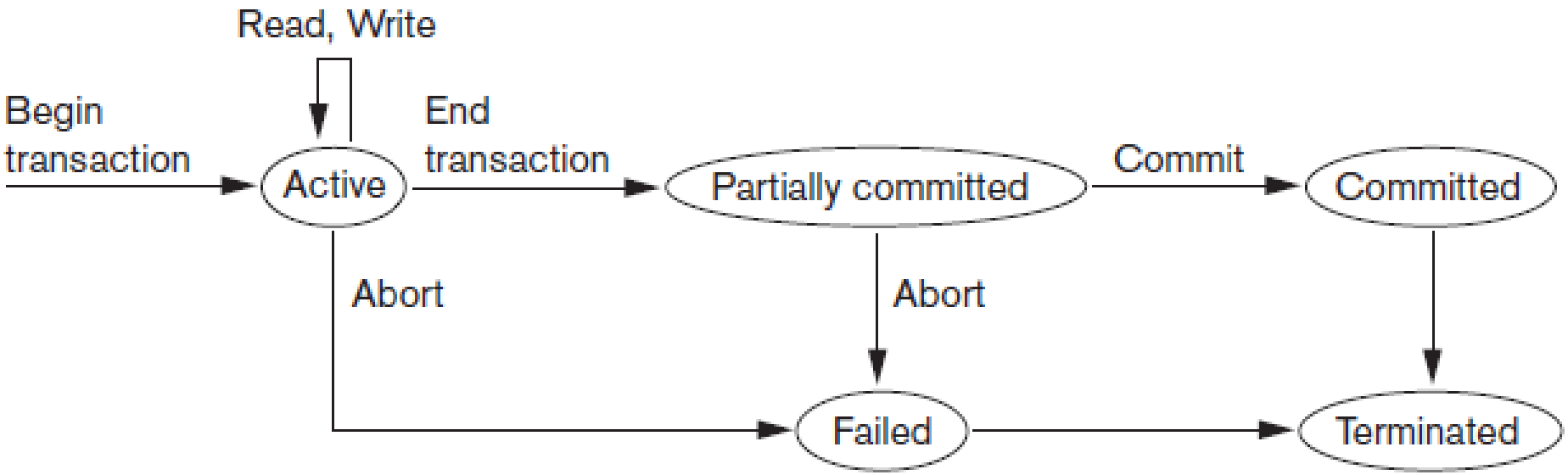
Transaction Processing

- Transaction processing systems are systems with large databases and hundreds of concurrent users executing database transactions.
- Examples : airline reservations, banking, credit card processing, online retail purchasing, stock markets, supermarket checkouts, and many other applications.
- These systems require high availability and fast response time for hundreds of concurrent users.

Transaction Processing

- A transaction is an executing program that forms a logical unit of database Processing.
- A transaction includes one or more database access operations—these can include insertion, deletion, modification, or retrieval operations.
- The database operations that form a transaction can either be embedded within an application program or they can be specified interactively via a high-level query language such as SQL.

Transaction States



Transaction States

- A transaction is an atomic unit of work that should either be completed in its entirety or not done at all.
- For recovery purposes, the system needs to keep track of when each transaction starts, terminates, and commits or aborts
- The recovery manager of the DBMS needs to keep track of the following operations:
- BEGIN_TRANSACTION: This marks the beginning of transaction execution.

Transaction States

- READ or WRITE: These specify read or write operations on the database items that are executed as part of a transaction.
- END_TRANSACTION: This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution.
- COMMIT_TRANSACTION: This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.

Transaction States

- A transaction goes into an active state immediately after it starts execution, where it can execute its READ and WRITE operations.
- When the transaction ends, it moves to the partially committed state. At this point, some recovery protocols need to ensure that a system failure will not result in an inability to record the changes of the transaction permanently.
- Once this check is successful, the transaction is said to have reached its commit point and enters the committed state.
- When a transaction is committed, it has concluded its execution successfully and all its changes must be recorded permanently in the database, even if a system failure occurs.

Transaction States

- A transaction can go to the failed state if one of the checks fails or if the transaction is aborted during its active state. The transaction may then have to be rolled back to undo the effect of its WRITE operations on the database.
- The terminated state corresponds to the transaction leaving the system.The transaction information that is maintained in system tables while the transaction has been running is removed when the transaction terminates. Failed or aborted transactions may be restarted later—either automatically or after being resubmitted by the user—as brand new transactions.

ACID Properties

- A transaction is a very small unit of a program and it may contain several lowlevel tasks.
- A transaction in a database system must maintain Atomicity, Consistency, Isolation, and Durability − commonly known as ACID properties − in order to ensure accuracy, completeness, and data integrity.

ACID Properties

Atomicity –

- This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none.
- There must be no state in a database where a transaction is left partially completed.
- States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.

ACID Properties

Consistency –

- The database must remain in a consistent state after any transaction.
- No transaction should have any adverse effect on the data residing in the database.
- If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.

ACID Properties

Durability –

- The database should be durable enough to hold all its latest updates even if the system fails or restarts.
- If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data.
- If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.

ACID Properties

Isolation –

- In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system.
- No transaction will affect the existence of any other transaction.